Zephyr Project

IT-Schulungen.com
Technology & Transformation

MCUboot-Architektur und Imageformat

Bootkette, Slots, Metadaten und Updatezustände

5.05

MCUboot trennt Bootentscheidung und Anwendung

BOOTKETTE

```
ROM / STARTSTUFE → MCUboot → ZEPHYR-IMAGE
```

BOOTLOADER	prüft Imagezustand und wählt den Startkandidaten
ANWENDUNG	wird als separates, austauschbares Image behandelt
METADATEN	Header und TLVs liefern Version, Hash und Signatur
TRAILER	hält Test-, Bestätigungs- und Swap-Status

MCUboot entscheidet anhand von Imageinhalt, Slotstatus und Updateverfahren.

Primary und Secondary Slot

FLASH-AUFTEILUNG

```
PRIMARY SLOT     startbares Image
SECONDARY SLOT   Updatekandidat
```

01 Primary Slot

Enthält das aktive oder zu prüfende Start-Image.

02 Secondary Slot

Nimmt ein neues Image für den Updateprozess auf.

03 Scratch / Status

Wird je nach Verfahren für Transfer und Fortschritt genutzt.

Slotgrößen, Ausrichtung und Trailerbereich müssen zum Updateverfahren passen.

Updateverfahren im Vergleich

Fünf Entscheidungen prägen Transfer, Platzbedarf und Startadresse

01

Swap

Images werden zwischen den Slots ausgetauscht

02

Move

Image wird schrittweise in den Primary Slot verschoben

03

Overwrite

Primary wird durch den Updatekandidaten ersetzt

04

Direct-XIP

Image startet direkt aus dem gewählten Slot

05

Auswahl

Verfahren passend zu Flashlayout und Revert-Bedarf wählen

Das Verfahren bestimmt Layout, Trailerbedarf, Revert-Fähigkeit und Laufzeitkosten.

Single-Image und Multi-Image

SINGLE IMAGE

Ein Image verwalten

Ein Primary-/Secondary-Paar für die Anwendung.

MULTI IMAGE

Mehrere Images koppeln

Mehrere Slotpaare mit koordinierter Auswahl.

ABHÄNGIGKEIT

Kompatibilität prüfen

Dependency-TLVs benennen erforderliche Imageversionen.

Multi-Image-Updates benötigen eine konsistente Entscheidung über alle beteiligten Images.

Der MCUboot-Imageheader

01	`Identität`	Magic kennzeichnet ein gültiges MCUboot-Imageformat
02	`Speicherlage`	Load Address, Header Size und Image Size beschreiben das Layout
03	`Eigenschaften`	Flags und Image Version steuern Interpretation und Auswahl

Der Header liegt vor dem Payload und wird in die Integritätsprüfung einbezogen.

Headerfelder im Überblick

IMAGE HEADER

```
magic | load_addr | hdr_size | img_size | flags | version
```

01 **Größen und Adressen**

grenzen Header, Payload und Zielbereich eindeutig ab.

02 **Version und Flags**

unterstützen Auswahl, Kompatibilität und Ladeverfahren.

Feldwerte müssen zum erzeugten Image, Slotlayout und Bootmodus passen.

Type-Length-Values (TLVs) tragen Prüfinformationen

TLV-BEREICHE

```
PROTECTED TLV   gehasht
REGULAR TLV     Hash / Signatur / Schlüsselinfo
```

01 Protected TLV

Geschützte Metadaten fließen in den Image-Hash ein.

02 Regular TLV

Trägt Hash, Signatur und Informationen zum öffentlichen Schlüssel.

MCUboot prüft die kryptografischen TLVs, bevor ein Image als startbar gilt.

Image-Trailer steuert den Updatezustand

`Magic`	kennzeichnet einen vorbereiteten Trailer
`Swap Type`	beschreibt Test, Permanent oder Revert
`Copy Done`	markiert eine abgeschlossene Bildübertragung
`Image OK`	bestätigt das neue Image dauerhaft
`Statusfelder`	ermöglichen Wiederaufnahme nach Unterbrechung

Der Trailer macht den Updatefortschritt persistent und unterstützt Recovery nach Reset.

Vom Kandidaten zur dauerhaften Version

TEST

Probeweise

Update einmal starten; Anwendung bleibt zunächst unbestätigt.

CONFIRM

Dauerhaft

Anwendung bestätigt sich; Image OK wird gesetzt.

REVERT

Zurückrollen

Fehlende Bestätigung aktiviert wieder den bekannten Stand.

MCUboot kombiniert Header, TLVs und Trailer zu einer nachvollziehbaren Bootentscheidung.